

Informe técnico de avance

Implementación y validación del enlace USB–SPI full-duplex de la DAQ basada en dsPIC33CK64MC105

Haniel Ulises
Servicio social

6 de octubre de 2026

Resumen

Se informa el trabajo realizado entre el 1 y el 6 de octubre de 2026 para sustituir el protocolo de comunicación entre Simulink y la tarjeta de adquisición basada en el microcontrolador dsPIC33CK64MC105 y el puente USB–SPI FT2232H. Se implementó el esquema planteado en la propuesta: una transferencia full-duplex de ocho bytes por paso de simulación, delimitada por la señal de selección de esclavo, protegida con número de secuencia y CRC-8, y atendida en el dsPIC por el controlador DMA. Se desarrollaron, además, un programa de prueba del enlace para Windows y Linux, un visor en tiempo real, un generador de reportes y una batería de pruebas automatizada. Las campañas de prueba sobre la tarjeta permitieron identificar y corregir dos causas de error sucesivas. Con el reloj de 4 MHz, la latencia del SPI esclavo y la duración de la interrupción de fin de trama dañaban la mayoría de las respuestas; ambos efectos se eliminaron al activar el PLL ($F_{cy} = 50$ MHz). Quedó una tasa residual de entre 0.025 % y 1 % de transferencias dañadas, cuyo origen se determinó como pulsos falsos en la señal CS causados por la ausencia de una tierra común entre las tarjetas. Con un cable de GND entre la Curiosity Nano y el FT2232H, las cuatro pruebas de 100 000 tramas concluyeron sin errores. La salida analógica respondió a una rampa de ± 1 V sin errores de comunicación; queda pendiente su verificación con osciloscopio, la conexión del motor y del encoder, y la validación del bloque de Simulink.

Índice

1. Introducción	3
1.1. Antecedentes	3
1.2. Objetivos	3
1.3. Estado general	3
2. Descripción del sistema implementado	3
2.1. Componentes	3
2.2. Conexiones	4
2.3. Protocolo de comunicación	4
3. Firmware dsPicDaq	5
3.1. Enlace SPI con DMA	5
3.2. Fin de trama	6
3.3. Vigilancia	6
3.4. Reloj del sistema	6
3.5. Causa de reinicio	6

3.6. Salida analógica	6
3.7. Compilación	6
4. Software en la PC	7
4.1. Biblioteca del protocolo y pruebas sin hardware	7
4.2. Programa de prueba del enlace	7
4.3. Visor en tiempo real	7
4.4. Reportes y batería de pruebas	7
4.5. Bloque de Simulink	8
5. Metodología de prueba	8
5.1. Clasificación de las respuestas	8
5.2. Criterio de aceptación	8
5.3. Análisis de las respuestas dañadas	8
6. Resultados	8
6.1. Lazo interno del FT2232H	9
6.2. Firmware con $F_{cy} = 4$ MHz	9
6.3. Firmware con PLL ($F_{cy} = 50$ MHz)	9
6.4. Ambas tarjetas en un solo hub USB	10
6.5. Cable de tierra entre las tarjetas	10
7. Diagnóstico de los pulsos falsos en CS	11
7.1. Evidencia	11
7.2. Mecanismo	12
7.3. Causa	12
8. Cambios derivados	12
8.1. Vigilancia al inicio de una prueba	12
8.2. Documentación	12
9. Salida analógica y conexión de la planta	13
9.1. Prueba de la rampa	13
9.2. Niveles de la salida	13
9.3. Motor y encoder	13
10. Trabajo pendiente y recomendaciones	14
A. Historial de cambios	14
B. Ubicación de los resultados	15

1. Introducción

1.1. Antecedentes

El sistema DAQ-SPI emula una tarjeta de adquisición de datos para el control de un motor de corriente directa desde Simulink. La PC se comunica por USB con un FT2232H que opera como maestro SPI; el dsPIC33CK64MC105, montado en una tarjeta Curiosity Nano, actúa como esclavo, escribe la salida analógica en un DAC8554 y cuenta los pulsos del encoder con su módulo QEI.

En la versión anterior, la escritura al DAC y la lectura del encoder se realizaban con dos transacciones independientes (`SPI_Write` y `SPI_Read`) desde tres S-Functions. Como se documentó en la propuesta ([docs/propuesta.pdf](#)), ese esquema utilizaba el bus como un canal half-duplex: cada transacción introducía o descartaba bytes que el otro extremo no esperaba, las FIFOs de cuatro niveles del SPI esclavo se desbordaban, la bandera `SPIROV` detenía la recepción de forma permanente y la trama no se delimitaba con `CS`. Además, al terminar la simulación se enviaba el código `0x0000`, que corresponde a -2.5 V , de modo que el motor quedaba energizado.

1.2. Objetivos

1. Implementar el protocolo propuesto: una sola transferencia full-duplex de longitud fija por paso de simulación, con detección de errores.
2. Reescribir el firmware del dsPIC para que el movimiento de datos del SPI no dependa de la CPU.
3. Disponer de herramientas que permitan medir el enlace sin Simulink, con un criterio de aceptación de cero errores en al menos 100 000 tramas.
4. Sustituir las tres S-Functions por un solo bloque de Simulink.
5. Validar la salida analógica y la lectura del encoder con el motor.

1.3. Estado general

La tabla 1 resume el avance respecto al plan de validación de la propuesta.

Etapas	Alcance	Estado
1. Firmware	dsPicDaq: SPI1 con DMA, PLL, vigilancia, causa de reinicio	Concluida
2. Prueba del enlace	prueba_enlace, visor, reportes; 100 000 tramas sin errores	Concluida
3. Salida analógica	Rampa de $\pm 1\text{ V}$ enviada sin errores; falta osciloscopio	En curso
4. Simulink	S-Function daqPic escrita; falta compilarla y probarla en Windows	Pendiente
5. Pruebas con el motor	Motor y encoder identificados; falta etapa de potencia	Pendiente

Tabla 1: Estado de las etapas al 6 de octubre de 2026.

2. Descripción del sistema implementado

2.1. Componentes

- **PC.** Windows 11 con MATLAB/Simulink para la operación, y Linux (Fedora 43) para el desarrollo y las pruebas del enlace.

- **Puente USB–SPI.** FT2232H, canal A en modo MPSSE, maestro SPI en modo 0 a 1 MHz, con CS activo en bajo en ADBUS3. Su EEPROM está reprogramada con el identificador 2099:0001 (“Custom DAC USB Bridge”, CINVESTAV).
- **Microcontrolador.** dsPIC33CK64MC105 en la tarjeta Curiosity Nano, con depurador nEDBG y programación por arrastre a la unidad CURIOSITY.
- **Convertidor digital-analógico.** DAC8554 de Texas Instruments (cuatro canales, 16 bits), en encapsulado TSSOP-16 montado sobre un adaptador Proto-Advantage PA0017C. Se utiliza el canal A (VOUTA, terminal 1).
- **Planta.** Motorreductor Pittman GM8724S017-R1 de 14 V con encoder incremental de 500 ciclos por revolución.

2.2. Conexiones

La tabla 2 reproduce las conexiones vigentes. La última fila, el cable de tierra entre las tarjetas, se agregó como resultado de este trabajo (sección 7).

Señal	dsPIC	FT2232H	DAC8554
SCK1 (entrada)	RB13	AD0 (SCK)	
SDI1	RB14	AD1 (MOSI)	
SDO1	RB11	AD2 (MISO)	
SS1 / INT1	RB10	AD3 (CS)	
SCK2	RB0		terminal 10 (SCLK)
SDO2	RB2		terminal 11 (DIN)
SYNC (GPIO)	RB4		terminal 9 (SYNC)
QEI A / B	RB8 / RB9		
LED0	RD10		
GND	GND (junto a RB10)	GND	terminal 6 (GND)

Tabla 2: Conexiones del sistema.

2.3. Protocolo de comunicación

Cada paso de simulación ejecuta una transferencia de ocho bytes en ambas direcciones (tabla 3). La respuesta que el dsPIC envía en la transferencia k se construyó al terminar la transferencia $k - 1$; por ello el byte 1 de la respuesta es el eco del número de secuencia de la última trama válida recibida, lo que permite a la PC detectar tramas perdidas. El CRC-8 utiliza el polinomio 0x07 (CRC-8/SMBUS) y se calcula por tabla. El formato, las constantes y la conversión de voltaje a código se definen en un solo archivo, `protocolo/daq_protocolo.h`, que comparten el firmware y todos los programas de la PC.

Byte	PC → dsPIC (MOSI)		dsPIC → PC (MISO)	
0	0xA5	inicio de trama	0x5A	inicio de trama
1	seq	número de secuencia	seq	eco de la última trama válida
2	dac_h	valor del DAC, byte alto	enc0	posición del encoder
3	dac_l	valor del DAC, byte bajo	enc1	(entero de 32 bits
4	flags	banderas de control	enc2	con signo,
5	—	reservado	enc3	<i>little-endian</i>)
6	—	reservado	status	estado del dsPIC
7	crc	CRC-8 de los bytes 0 a 6	crc	CRC-8 de los bytes 0 a 6

Tabla 3: Formato de trama.

El valor del DAC se obtiene como código = $13107(V + 2.5)$, con saturación a ± 2.5 V; un valor no numérico se lleva a 0 V. El código 32 767 corresponde a 0 V. El byte **flags** habilita la salida analógica (bit 0; con cero el DAC se lleva a 0 V) y ordena poner en cero el contador del encoder (bit 1). El byte **status** se describe en la tabla 4; los cuatro bits altos se agregaron durante este trabajo para reportar la causa del último reinicio del dsPIC.

Bits	Nombre	Significado
0	ERR_CRC	la última trama recibida tuvo CRC incorrecto
1	ERR_INICIO	la última trama recibida no comenzó con 0xA5
2	VIGILANCIA	transcurrieron 50 ms sin una trama válida; el DAC se llevó a 0 V
3	ERR_LONGITUD	la última transferencia no tuvo exactamente ocho bytes
4–7	REINICIO	causa del arranque (encendido, bajo voltaje, MCLR, watchdog, software, opcode ilegal, configuración o alguna de las trampas); se envía una sola vez, en la primera respuesta después del arranque

Tabla 4: Campos del byte **status**.

3. Firmware dsPicDaq

El firmware se escribió a partir del proyecto anterior **dsPicController**, cuya configuración generada por MCC se conserva para los pines y periféricos que no cambian. Se encuentra en MPLAB/dsPicDaq.X y su imagen compilada en MPLAB/dsPicDaq.hex.

3.1. Enlace SPI con DMA

SPI1 opera como esclavo en modo 0 ($CKP = 0$, $CKE = 1$), con la entrada de selección **SS1** habilitada y en modo de buffer estándar. La FIFO del modo mejorado, de cuatro niveles en 8 bits, no alcanza para una trama, por lo que no aporta en este esquema. Dos canales del controlador DMA atienden el módulo:

- **DMA0** (disparo 0x02, SPI1 RX): copia SPI1BUFL a `rx_dma[]` con cada byte recibido.
- **DMA1** (disparo 0x03, SPI1 TX): copia `tx_dma[1..7]` a SPI1BUFL cada vez que el buffer de transmisión queda vacío. El primer byte se escribe directamente al armar la trama.

Los eventos de SPI1 se usan sólo como disparo del DMA; las interrupciones de CPU del módulo permanecen deshabilitadas. Durante la transferencia la CPU no interviene.

3.2. Fin de trama

CS (RB10) se lleva también a la entrada INT1 mediante PPS. La interrupción, configurada en el flanco de subida, ejecuta la secuencia siguiente:

1. Deshabilita ambos canales de DMA.
2. Valida la trama: el canal de RX debe haber terminado, el buffer de recepción debe estar vacío y sin desbordamiento (exactamente ocho bytes); después se revisan el encabezado y el CRC. Una trama válida actualiza el valor del DAC, el eco de secuencia y reinicia la vigilancia.
3. Reinicia SPI1 deshabilitándolo y habilitándolo, lo que vacía los buffers y borra `SPIROV`, de modo que cada trama comienza alineada.
4. Lee el contador del QEI y construye la respuesta siguiente.
5. Programa de nuevo los dos canales de DMA.

La terminal RD10 permanece en alto mientras se ejecuta la rutina, lo que permite medir su duración con un analizador lógico.

3.3. Vigilancia

Timer1 genera una interrupción cada 50 ms; cada trama válida lo reinicia. Si expira, el DAC se lleva a 0 V y se reporta la bandera `VIGILANCIA` una sola vez. Sólo una trama válida restablece la operación. Con esto, si Simulink se detiene o la PC deja de enviar tramas, el motor se desenergiza.

3.4. Reloj del sistema

Los firmwares anteriores operaban sin PLL ($F_{osc} = 8\text{ MHz}$, $F_{cy} = 4\text{ MHz}$). Las primeras pruebas mostraron que ese reloj era insuficiente (sección 6.2), por lo que `reloj.c` configura el oscilador FRC de 8 MHz con el PLL primario para obtener $F_{osc} = 100\text{ MHz}$ y $F_{cy} = F_P = 50\text{ MHz}$. Con el cambio se recalculó el periodo de la vigilancia y se dividió entre 16 el filtro digital del QEI, para conservar el rechazo de pulsos de alrededor de $1\text{ }\mu\text{s}$.

3.5. Causa de reinicio

`reinicio.c` lee el registro `RCON` al arrancar y sustituye el manejador de trampas de MCC para guardar, en RAM persistente, el código de la trampa antes de reiniciar. La causa se envía en los cuatro bits altos del `status`. Así, un reinicio del dsPIC durante la operación ya no pasa inadvertido para la PC.

3.6. Salida analógica

El lazo principal escribe en el DAC8554 por SPI2 sólo cuando la interrupción deposita un valor nuevo. Cada escritura es una trama de 24 bits enmarcada por `SYNC`: el byte de control `0x10` (dirección 0, canal A, actualización inmediata) seguido del código de 16 bits.

3.7. Compilación

Además del proyecto de MPLAB X, el script `compilar.sh` compila el firmware en Linux con XC-DSC v4.00 y el paquete `dsPIC33CK-MC_DFP`, con las mismas opciones del proyecto, y deja el `.hex` en `MPLAB/`. La imagen se carga copiándola a la unidad `CURIOSITY`, tanto en Windows como en Linux.

4. Software en la PC

4.1. Biblioteca del protocolo y pruebas sin hardware

`protocolo/prueba_protocolo.c` prueba el formato, el CRC, la conversión de voltaje y un modelo del esclavo que reproduce la vigilancia, la acumulación de errores en el `status`, la vuelta del número de secuencia y el reporte de reinicio. Estas pruebas, junto con la compilación de las herramientas, se ejecutan en GitHub Actions con gcc y clang.

4.2. Programa de prueba del enlace

`herramientas/prueba_enlace` envía un número dado de tramas, lo más rápido posible o con un periodo fijo, y clasifica cada respuesta. Compila en Windows con libMPSSE-SPI y en Linux con libftdi1, mediante `mpsse_linux.h`, que implementa sobre libftdi1 el subconjunto de libMPSSE-SPI que se utiliza. Sus opciones principales son:

- `-n`: número de tramas; `-periodo`: periodo en microsegundos; `-reloj`: frecuencia de SCK.
- `-salida`: genera una rampa triangular de ± 1 V con periodo de 2000 tramas. Sin esta opción la salida permanece en 0 V.
- `-lazo` (sólo Linux): une MOSI con MISO dentro del FT2232H para probar el puente sin el dsPIC.
- `-registro`: guarda en un archivo CSV los tiempos, los bytes enviados y recibidos, el resultado y el `status` de cada trama.

Al terminar, el programa envía una trama con la salida deshabilitada, de modo que el DAC queda en 0 V. En Linux se requiere la regla `70-daq-spi.rules` para acceder al dispositivo sin privilegios de administrador.

4.3. Visor en tiempo real

`herramientas/visor_enlace` ejecuta la misma prueba con una interfaz gráfica al estilo del Scope de Simulink (Dear ImGui e ImPlot sobre GLFW/OpenGL): voltaje, posición y duración de cada transferencia con el eje de tiempo ligado, contadores de errores y de pasos atrasados, disparo por error, generador de señal, tabla de tramas con error y mapa de bits erróneos. Cuenta con un modo simulado (modelo del dsPIC y del motor) y con opciones para ejecutarse en lote y guardar una captura de la ventana. El núcleo de la prueba (`herramientas/comun/enlace_prueba.h`) es común al visor y a `prueba_enlace`, de modo que ambos clasifican igual.

4.4. Reportes y batería de pruebas

`herramientas/reporte/reporte.py` genera, a partir del CSV, un reporte en PDF (LaTeX y gnuplot) con la configuración, los errores por tipo, la duración de las transferencias, los errores acumulados y el mapa de bits erróneos; si existe un archivo de notas, lo incluye como sección de análisis. `herramientas/bateria_pruebas.sh` ejecuta en secuencia las cuatro pruebas de referencia (lazo interno y dsPIC, cada una lo más rápido posible y a 1 kHz, con 100 000 tramas) con `prueba_enlace` o con el visor, y genera los reportes en `resultados/<fecha>_<herramienta>/.`

4.5. Bloque de Simulink

SIMULINK/DaqPic/daqPic.cpp es una S-Function que sustituye a `initializePic`, `leerPic` y `escribirPic`. Abre el FT2232H, ejecuta una transferencia por paso en `mdlUpdate`, se sincroniza con el reloj de la PC, cuenta los errores de comunicación y los pasos atrasados, y al terminar deja la salida en 0 V. No tiene transmisión directa, por lo que puede usarse en lazo cerrado sin lazos algebraicos; la posición se entrega con un retardo de dos periodos. Al iniciar envía dos tramas: la primera pone en cero el encoder y la segunda verifica que el dsPIC responda con el protocolo nuevo.

5. Metodología de prueba

5.1. Clasificación de las respuestas

Cada respuesta se clasifica en la PC y se complementa con lo que reporta el dsPIC en el `status` de la respuesta siguiente (tabla 5). La primera respuesta de cada prueba se construyó antes de ésta y no se evalúa.

Clase	Criterio
USB / longitud	la llamada falló o no transfirió ocho bytes
Eco (sólo lazo)	los bytes recibidos difieren de los enviados
Encabezado	el byte 0 no es 0x5A
CRC	el CRC de la respuesta no coincide
Secuencia	el eco no corresponde a la trama anterior
dsPIC: CRC, encabezado, longitud	banderas del <code>status</code>
Vigilancia	bandera del <code>status</code> , salvo la que llega en la respuesta a la segunda trama, que proviene de la inactividad previa a la prueba
Reinicio	causa distinta de cero en el <code>status</code>

Tabla 5: Clasificación de las respuestas.

5.2. Criterio de aceptación

Conforme a la propuesta, el enlace se acepta cuando las cuatro pruebas de la batería (100 000 tramas cada una) terminan sin errores de ningún tipo. Se registran además la duración mínima, media y máxima de `SPI_ReadWrite`, que determina el periodo de muestreo alcanzable.

5.3. Análisis de las respuestas dañadas

Para localizar el origen de un error, cada respuesta dañada se compara con la respuesta esperada: el encabezado, el eco de la secuencia anterior, la posición del encoder y el `status`. Esto permite construir el mapa de bits erróneos y determinar en qué byte comienza la corrupción.

6. Resultados

Todas las pruebas se ejecutaron en Linux con SCK de 1 MHz y la salida analógica en 0 V. La tabla 6 resume las campañas; las secciones siguientes las describen en orden cronológico.

Campaña	Prueba	Encabezado	CRC (PC)	Secuencia	Longitud (dsPIC)
$F_{cy} = 4 \text{ MHz}$	máxima vel.	94.90 %	4.50 %	0.57 %	0.56 %
	1 kHz	0.13 %	83.67 %	0.16 %	0.07 %
PLL, puertos separados	máxima vel.	0	0.63 %	0.66 %	0.65 %
	1 kHz	0	0.68 %	0.73 %	0.72 %
PLL, visor, puertos sep.	máxima vel.	0	1.03 %	1.08 %	1.07 %
	1 kHz	0	1.04 %	1.11 %	1.10 %
PLL, un hub	máxima vel.	0	0	0	0
	1 kHz	0	0	0	0
PLL, un hub, visor	1 kHz	0	0.025 %	0.027 %	0.026 %
PLL, un hub, cable GND, visor	máxima vel.	0	0	0	0
	1 kHz	0	0	0	0

Tabla 6: Proporción de tramas con error, 100 000 tramas por prueba. Las pruebas en lazo interno del FT2232H no presentaron errores en ninguna campaña.

6.1. Lazo interno del FT2232H

Se verificó primero el lado de la PC sin el dsPIC. En las tres campañas en que se ejecutó, cada uno de los 800 000 bytes de cada prueba regresó intacto por el lazo MOSI $\rightarrow$ MISO. Ocho bytes a 1 MHz ocupan $64 \mu\text{s}$ en el bus y la transferencia más corta midió $78 \mu\text{s}$, por lo que el costo del USB y de libftdi es de unos $15 \mu\text{s}$. La duración media fue de 114 a $154 \mu\text{s}$; las transferencias de varios milisegundos son esporádicas y se deben a la planificación del sistema operativo, que no es de tiempo real.

6.2. Firmware con $F_{cy} = 4 \text{ MHz}$

El 5 de octubre se probó el firmware con el reloj original. Se encontraron tres defectos:

1. **Errores de encabezado a máxima velocidad (94.9 %).** La interrupción de fin de trama, estimada en unos $100 \mu\text{s}$ con este reloj, no terminaba de rearmar el DMA antes de la siguiente transferencia. Con una separación menor de $20 \mu\text{s}$ entre transferencias sólo el 4.5 % de las respuestas tenía encabezado correcto; con $100 \mu\text{s}$ o más, el 100 %.
2. **Errores de CRC a 1 kHz (83.7 %).** El bit 7 de los bytes 1, 2 y 7 concentraba los errores; en el 77 % de los casos el valor recibido era el del bit 0 del byte anterior. El SPI esclavo, sincronizado con un reloj de periférico de 4 MHz, no alcanzaba a presentar el primer bit de cada byte antes de que el FT2232H lo muestreara, 500 ns después. Con SCK de 500 kHz los errores de CRC bajaron al 0.55 %, lo que confirmó la causa.
3. **Respuestas de arranque (5.9 %).** Respuestas con secuencia y posición en cero, compatibles con reinicios del dsPIC por las trampas de MCC, que en producción ejecutan **reset**.

Estos resultados motivaron la activación del PLL y el reporte de la causa de reinicio.

6.3. Firmware con PLL ($F_{cy} = 50 \text{ MHz}$)

Con el PLL desaparecieron los errores de encabezado aun con la separación mínima entre transferencias, el bit 7 llegó siempre correcto, desaparecieron las respuestas de arranque y el dsPIC no reportó ningún reinicio. Persistió, sin embargo, un defecto en el 0.6–0.7 % de las transferencias: en

672 de las 675 respuestas con CRC incorrecto a 1 kHz, el dsPIC reportó en la respuesta siguiente que esa misma transferencia no tuvo ocho bytes, es decir, la falla dañaba ambos sentidos a la vez. La corrupción comenzaba en el byte 1 en el 63 % de los casos y en el byte 7 en el 26 %. Con el visor, que carga más la PC, la proporción subió al 1.0 %. En ese momento la hipótesis de trabajo fue un disparo de más o de menos en el canal de DMA de transmisión, y se planteó revisarlo con el analizador lógico.

6.4. Ambas tarjetas en un solo hub USB

El 6 de octubre las dos tarjetas se conectaron a un mismo hub USB, en lugar de a puertos distintos de la PC. Sin modificar el firmware, las cuatro pruebas con `prueba_enlace` terminaron sin errores de comunicación. Con el visor, en cambio, se registraron 25 respuestas con CRC incorrecto en 100 000 tramas a 1 kHz (0.025 %), con el mismo patrón de longitud incorrecta reportada por el dsPIC. El análisis de esas respuestas, descrito en la sección 7, permitió identificar la causa.

6.5. Cable de tierra entre las tarjetas

Se conectó un cable entre la terminal GND de la Curiosity Nano contigua a RB10 y la terminal GND de la tarjeta del FT232H. La batería completa, ejecutada con el visor (la condición más desfavorable), concluyó sin errores en las cuatro pruebas. La tabla 7 presenta las duraciones medidas en esa campaña.

Prueba	Tramas/s	Mín. (μs)	Media (μs)	Máx. (μs)	Atrasos
Lazo, máxima velocidad	7 622	79	127	4 808	—
Lazo, 1 kHz	1 000	87	154	3 896	128
dsPIC, máxima velocidad	7 408	81	130	5 260	—
dsPIC, 1 kHz	1 000	86	164	12 807	485

Tabla 7: Duración de `SPI_ReadWrite` con el cable de GND, medida con el visor en Linux (`resultados/2026-10-06_gnd`).

Los pasos atrasados corresponden a periodos en que la PC no inició la transferencia dentro de 1 ms. Aparecen también en el lazo interno, que no involucra al dsPIC, por lo que dependen de la carga de la PC y no del enlace. La figura 1 muestra la ventana del visor al terminar la prueba a 1 kHz.



Figura 1: Visor del enlace al terminar la prueba de 100 000 tramas a 1 kHz con el cable de GND: cero errores.

7. Diagnóstico de los pulsos falsos en CS

7.1. Evidencia

En la corrida con el visor y sin cable de GND, las respuestas dañadas no contienen bits aislados erróneos, sino el inicio de la respuesta repetido a mitad de la trama y recorrido algunos bits. La figura 2 presenta un caso.

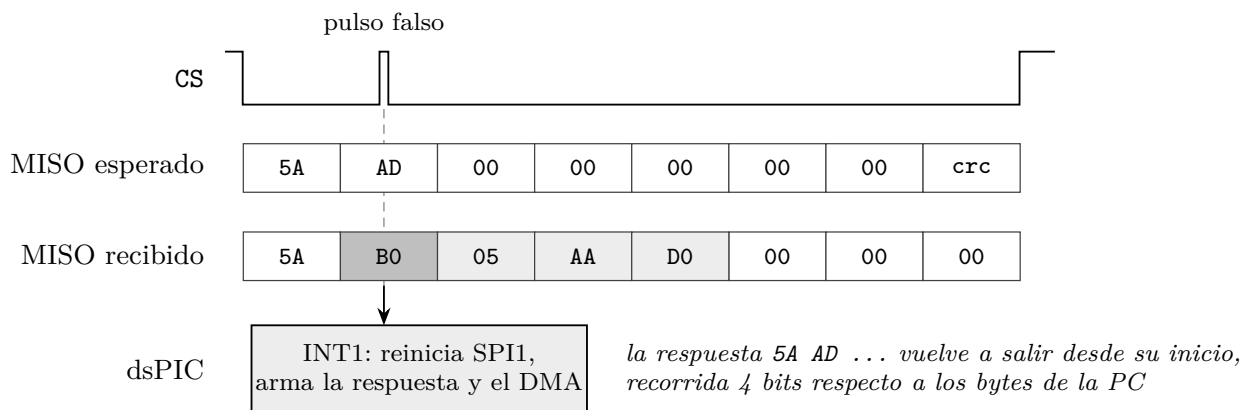


Figura 2: Respuesta dañada por un pulso falso en CS (resultados/2026-10-06_dock/visor_dspic_1khz). Se esperaba 5A AD 00 00 00 00 00 00 y se recibió 5A B0 05 AA D0 00 00 00: a partir del byte 1 reaparecen 5A AD desplazados medio byte.

En otra respuesta, 5A 40 0B 4B 80 00 00 00 en lugar de 5A 5C 00 ..., la secuencia de bits

01011010 01011100 (5A 5C) aparece a partir del tercer bit del byte 1. De las 25 respuestas dañadas, la corrupción comienza en el byte 1 en 13, en el byte 2 en una y en el byte 7 en 11, según el momento en que ocurre el pulso; es el mismo patrón observado en la campaña con puertos separados.

7.2. Mecanismo

El único código del firmware que vuelve a construir la respuesta desde su inicio es la interrupción INT1, que se dispara con el flanco de subida de CS. La secuencia es la siguiente:

1. Un pulso de ruido en CS se interpreta como fin de trama.
2. INT1 reinicia SPI1, construye la respuesta y rearma el DMA mientras la PC continúa la transferencia.
3. La PC recibe el resto de la respuesta desplazado algunos bits; el CRC no coincide.
4. Al subir CS al final de la transferencia, el dsPIC encuentra una trama incompleta y lo reporta con la bandera de longitud en la respuesta siguiente.

Este mecanismo explica las dos observaciones que la hipótesis del DMA no explicaba: que la falla dañe ambos sentidos a la vez y que la corrupción comience en bytes concretos.

7.3. Causa

Las tarjetas no compartían una tierra directa: la única referencia común de las señales del SPI era la tierra de los cables USB a través de la PC. Con cada tarjeta en un puerto distinto, la diferencia de potencial y el ruido entre ambas referencias eran suficientes para producir pulsos en CS en el 0.7 % de las transferencias. El hub acertó el camino de retorno y redujo la tasa a cero con `prueba_enlace`, pero no con el visor, cuyo uso de la GPU incrementa el ruido en la tierra de la PC. El cable de GND eliminó los errores en todas las condiciones, lo que confirma la causa.

8. Cambios derivados

8.1. Vigilancia al inicio de una prueba

Si el enlace permanece inactivo más de 50 ms antes de una prueba, el dsPIC reporta la bandera de vigilancia en la respuesta a la segunda trama. Ese aviso es correcto, pero no corresponde a un error durante la prueba, y hacía que la batería terminara con el resultado “con errores” sin que hubiera ninguno. Se agregó la función `status_contable()` en el núcleo común, que omite esa bandera en la trama 1; la usan `prueba_enlace`, el visor y, con el mismo criterio, el generador de reportes. El registro CSV conserva el `status` completo.

En `daqPic` ocurre lo mismo entre `mdlStart` y el primer paso de simulación, intervalo en el que Simulink inicializa el resto del modelo y que puede rebasar 50 ms. El bloque ignora ahora la vigilancia en la respuesta del segundo paso; sin este cambio, el indicador **Errores** comenzaría en 1 y la prueba del paso 4 no podría cumplir el criterio de aceptación. El cambio no se ha compilado, porque requiere MATLAB en Windows.

8.2. Documentación

- README.md: fila de GND en la tabla de conexiones y explicación del efecto de su ausencia.

- **PRUEBAS.md**: el cable de GND como requisito; la fila “pulsos falsos en CS” en la tabla de interpretación de errores; la carga y la compilación del firmware en Linux; el punto 3 de la verificación con analizador lógico (entrega del segundo byte por el DMA) se marca como comprobado, ya que no hubo errores de secuencia en 200 000 tramas.
- Notas de análisis en cada carpeta de **resultados/**, incluidas en los reportes en PDF.

9. Salida analógica y conexión de la planta

9.1. Prueba de la rampa

Se ejecutó el paso 3 del procedimiento con `prueba_enlace -n 10000 -periodo 1000 -salida: 10000` tramas a 1 kHz con una rampa triangular de ± 1 V (códigos de 19 660 a 45 874) y la salida habilitada en todas las tramas. La prueba terminó sin errores de comunicación y con la trama final de 0 V. La posición del encoder permaneció en cero, como corresponde, ya que el motor no estaba conectado. Falta observar **VOUTA** con osciloscopio.

9.2. Niveles de la salida

El DAC8554 sólo produce voltajes positivos:

$$V_{\text{OUTA}} = V_{\text{REFL}} + \frac{\text{código}}{65536} (V_{\text{REFH}} - V_{\text{REFL}}).$$

La escala del protocolo (13 107 códigos por volt) corresponde a un intervalo de 5 V; con $V_{\text{REFL}} = 0$ y $V_{\text{REFH}} = 5$ V, el “0 V” del protocolo son 2.5 V reales en la terminal 1 y la rampa de ± 1 V debe observarse entre 1.5 y 3.5 V, con periodo de 2 s. La tabla 8 indica las terminales del adaptador PA0017C, que conserva la numeración del circuito.

Term.	Nombre	Conexión	Term.	Nombre	Conexión
1	VOUTA	etapa de potencia	16	LDAC	GND
2	VOUTB	sin uso	15	ENABLE	GND
3	VREFH	5 V	14	A1	GND
4	AVDD	alimentación	13	A0	GND
5	VREFL	GND	12	IOVDD	alimentación
6	GND	GND	11	DIN	RB2
7	VOUTC	sin uso	10	SCLK	RB0
8	VOUTD	sin uso	9	SYNC	RB4

Tabla 8: Terminales del DAC8554 y conexión esperada según el firmware. Fuente de la distribución: hoja de datos SLAS431B de Texas Instruments.

9.3. Motor y encoder

El motorreductor Pittman GM8724S017-R1 es de 14 V con escobillas. Su encoder de 500 ciclos por revolución produce, con el conteo en cuadratura del QEI, 2000 cuentas por revolución del motor. Para su conexión se consideran los puntos siguientes:

- **Etapa de potencia.** El DAC no puede alimentar el motor. Se requiere un amplificador con entrada analógica que reste el nivel de 2.5 V y entregue una tensión bipolar de hasta ± 14 V con la corriente del motor; un puente H con entrada PWM no es adecuado sin una etapa adicional. La etapa de potencia aparece como “por confirmar” en la arquitectura y debe identificarse.

- **Encoder.** En otros encoders Pittman de 500 CPR el conector de cinco terminales es: 1 GND, 2 índice, 3 canal A, 4 +5 V, 5 canal B. Este dato no se ha confirmado para el modelo instalado. El encoder requiere 5 V; los canales A y B se conectan a RB8 y RB9.
- **Niveles lógicos.** RB8 y RB9 comparten función con entradas analógicas (AN10, AN11) y con la programación (PGD1, PGC1). Mientras no se confirme en la hoja de datos que toleran 5 V, se recomienda un divisor (2.2 k Ω en serie y 3.3 k Ω a GND) en cada canal.

10. Trabajo pendiente y recomendaciones

1. Observar **VOUTA** con osciloscopio durante la rampa del paso 3.
2. Identificar la etapa de potencia del motor y verificar que compense el nivel de 2.5 V antes de conectar el motor.
3. Confirmar la distribución del conector del encoder y los niveles admisibles en RB8 y RB9; probar el encoder girando el eje a mano con el visor.
4. Compilar **daqPic** en Windows, ejecutar el modelo de prueba y verificar que **Errores** y **Atrasos** terminen en cero.
5. Repetir la medición de tiempos en Windows con la PC sin carga adicional, para fijar el periodo de muestreo mínimo.
6. Medir con analizador lógico la duración de la interrupción de fin de trama en RD10.
7. Opcional: rechazar en INT1 los flancos en que **CS** ya volvió a nivel bajo, como protección adicional en caso de que el cable de GND se desconecte. No es necesario con la conexión actual.
8. Realizar la prueba en contra con las tarjetas en puertos separados y el cable de GND, para documentar que la conexión USB deja de influir.

A. Historial de cambios

Fecha	Commit	Descripción
2026-10-01	b0176e2	Propuesta de solución al problema de concurrencia SPI y diagrama de arquitectura.
2026-10-02	fae8880	Documentación del DAC8554 y redacción del README.
2026-10-02	a89bc2e	Protocolo full-duplex, firmware dsPicDaq con DMA, prueba_enlace , S-Function daqPic y procedimiento de pruebas.
2026-10-03	7e7edd1	Conversión de voltaje a código centralizada en el protocolo.
2026-10-03	d4d7719	Modelo de la vigilancia y ampliación de las pruebas del protocolo.
2026-10-03	47ca3fd	Integración continua con gcc y clang.
2026-10-05	2b8c056	prueba_enlace en Linux con libftdi1 y lazo interno.
2026-10-05	4c9e9ea	Registro por trama, reportes en PDF y primeras pruebas con la tarjeta.
2026-10-05	391c072	PLL ($F_{cy} = 50$ MHz) y reporte de la causa de reinicio.

Fecha	Commit	Descripción
2026-10-05	0f01466	Visor en tiempo real y batería de pruebas.
2026-10-05	3099948	Resultados con PLL y con el visor.
2026-10-06	8cdfc55	Resultados con ambas tarjetas en un hub.
2026-10-06	1b8b3d2	Cable de GND, diagnóstico de los pulsos en CS y vigilancia inicial.

B. Ubicación de los resultados

Carpeta	Contenido
<code>resultados/2026-10-05</code>	$F_{cy} = 4$ MHz; incluye SCK de 250 y 500 kHz
<code>resultados/2026-10-05_pll</code>	PLL, puertos separados, <code>prueba_enlace</code>
<code>resultados/2026-10-05_visor</code>	PLL, puertos separados, visor
<code>resultados/2026-10-06_dock</code>	PLL, un hub, <code>prueba_enlace</code> ; corrida del visor sin GND
<code>resultados/2026-10-06_gnd</code>	PLL, un hub, cable de GND, visor
<code>resultados/2026-10-06_salida</code>	Rampa de ± 1 V (paso 3)

Cada carpeta contiene, por prueba, la salida del programa (`.txt`), el reporte (`.pdf`) y, cuando existen, las notas de análisis (`_notas.tex`) y la captura del visor (`.png`).